

Note that this assignment was done assuming $temp += x * y$ is 2 executions, one for the multiplication one for addition

1

A

Pseudo

```
Fetch_memory X[0:N-1]
Fetch_memory a
```

```
temp = 0
```

```
for(i=0 to N-1) {
    Fetch_cache a
    Fetch_cache X[i]
    temp += a * X[i]
}
```

```
write temp to y
```

Performance

Initial: $10 + N + 10 + 1 = N + 21$ ns

For loop: $4 * N * 0.5$ ns

Write: $10 + 1$ ns

Total performance: $3N + 32$ ns

Data Reuse Factor

$Computations = 2N$

$Transfers = N + 2$

$Reuse = \frac{Computations}{Transfers} = \frac{2N}{N+2}$

B

Pseudo

```
Fetch_memory X[0:N-1]
Fetch_memory a

// Assume y = 0

for(i=0 to N-1) {
    Fetch_cache a
    Fetch_cache X[i]
    Fetch_memory y to register
    y += a * X[i]
    write y to Memory
}
```

Performance

Initial: $10 + N + 10 + 1 = N + 21$ ns

Loop: $N(0.5 + 0.5 + 10 + 1 + 0.5 + 0.5 + 10 + 1) = N(24.5)$ ns

Total performance: $25N + 21$ ns

Data Reuse Factor

$Computations = 2N$

$Transfers = 3N + 1$ (from X , and N both reads and writes for y)

$Reuse = \frac{Computations}{Transfers} = \frac{2N}{3N+1}$

C

Pseudo

```
Fetch_memory X[0:N-1]

// Assume y = 0

for(i=0 to N-1) {
```

```

    Fetch_cache X[i]
    Fetch_memory a to register
    Fetch_memory y to register
    y += a * X[i]
    write y to Memory
}

```

Performance

Initial: $10 + N = N + 10$ ns

Loop: $N(0.5 + 10 + 1 + 10 + 1 + 0.5 + 0.5 + 10 + 1) = N(34.5)$ ns

Total performance: $35.5N + 10$ ns

Data Reuse Factor

$Computations = 2N$

$Transfers = 4N$ (from X , and N reads for a and N both reads and writes for y)

$Reuse = \frac{Computations}{Transfers} = \frac{2N}{4N} = \frac{1}{2}$

2

A

B

C

D

E

F

3

A

In row major order we can fetch with $A[i][j : j + 1]$

Thus we get the following fetches:

1. $A[0][1] = 1$
2. $A[1][0 : 1] = 2, 3$
3. $A[1][2 : 3] = 4, 5$
4. $A[3][0] = 6$
5. $A[3][3] = 7$

For a total of 5 fetches

B

In column major order we can fetch with $A[i : i + 1][j]$

Thus we get the following fetches:

1. $A[0 : 1][1] = 1, 3$
2. $A[1][0] = 2$
3. $A[1][2] = 4$
4. $A[1][3] = 5$
5. $A[3][0] = 6$
6. $A[3][3] = 7$

For a total of 6 fetches

4

K psuedo code

```
for (i = 0 to N/K-1) {
  for (j = 0 to N/K-1) {

    // initialize K x K block of C to zero
    for (r = 0 to K-1) {
      for (c = 0 to K-1) {
        C[i*K + r][j*K + c] = 0;
      }
    }

    for (p = 0 to K-1) {

      // Fetch K rows of A
      for (r = 0 to K-1) {
        Fetch A[i*K + r][p*N/K : (p+1)*N/K];
      }

      // Fetch K cols of B
      for (c = 0 to K-1) {
        Fetch B[p*N/K : (p+1)*N/K][j*K + c];
      }

      // Compute K x K dot products of length N/K
      for (r = 0 to K-1) {
        for (c = 0 to K-1) {
          C[i*K + r][j*K + c] += A[i*K + r][p*N/K : (p+1)*N/K].B[p*N/K : (p+1)*N/K][j*K + c]
        }
      }
    }
  }

  // Store the K x K block of C back to memory
  for (r = 0 to K-1) {
    for (c = 0 to K-1) {
      Store C[i*K + r][j*K + c];
    }
  }
}
```

```

    }
}
}

```

Data Reuse for this Algorithm (N and K)

$$Computations = 2 * N/K * N/K * K * K * K * N/K = 2 * N^3$$

$$Transfers = N^2/K^2 * K * N + N^2/K^2 * K * N + 2 * N^2/K^2 * K^2 = 2N^3/K + 2N^2$$

$$Reuse = \frac{Computations}{Transfers} = \frac{2N^3}{2N^3/K + 2N^2} = \frac{2N}{2N/K + 2}$$

For K=2 Same as in Class?

For $K = 2$ then we get data reuse factor as $\frac{2N}{N+2}$ which we got on slide 37 of lecture 4.

$K=\sqrt{N}$ Have same Data Reuse as Block Matrix Multiplication?

For $K = \sqrt{N}$ then we get a data reuse factor as $\frac{2N}{2N/\sqrt{N}+2} = \frac{2N}{2\sqrt{N}+2}$. This is really close to block matrix multiplication which had a data reuse factor of $O(\sqrt{N})$, the only difference being the +2 in the denominator from the read and write to C in our algorithm.

K=1 Data Reuse Factor and What is this Algorithm?

Standard Naive matrix multiplication, row column and their dot product.

$$\text{Data reuse being } \frac{2N}{2N+2} = \frac{N}{N+2}$$

5

We can model a similar algorithm where instead of each thread being in charge of a single summation between two elements and writing the sum to the latter index. Copy the same recursive doubling algorithm but instead of each thread computing a summation, we do a comparison between the numbers, and make it so the left indexed element is the smaller one and the right is the bigger one (if this isn't the case we do a nice swap using a temp variable). This effectively performs a parallel bubble sort that moves the smallest element to index 0 and the largest element to the last index.

6

Slightly modified from slides:

```
// In thread bid, tid

// From Sldies
RMA: Read A[bid][tid] into cache
RMB: Read B[tid] into cache
Syncthreads()
RA: Read A[bid][tid] into processor
RB: Read B[tid] into processor
M: Temp[tid] <- Mult A[bid][tid]*B[tid]
Syncthreads()

// Recursive Doubling part
for (k = 1 to log_2(N)) {
    if ((tid+1) % 2^k == 0) {
        Temp[tid] = Temp[tid] + Temp[tid - 2^(k-1)];
    }
    Syncthreads()
}

if (tid == N - 1) {
    SMC: Store Temp[N-1] -> C[bid]
}
```